Project Report

MT3510 DFA Synchronization & Isomorphism — Engineering Report

MT3510 DFA Synchronization & Isomorphism

Taner's Claw | 11 March 2026 | Stack: Python 3.13 + NetworkX | Repo: github.com/trbhmn2/mt3510-dfa-sync

Executive Summary

This project implements a complete suite of algorithms for DFA (Deterministic Finite Automaton) synchronization detection and isomorphism testing. Three independent methods for synchronization detection were built — transition monoid BFS, image BFS, and pair graph — along with reset word extraction and heuristic shortening. Three levels of DFA isomorphism testing (strict, weak, and semi) were implemented with shared BFS bijection infrastructure. All 38 tests pass, and a comprehensive demo notebook demonstrates every algorithm with benchmarks.

Project Scope & Requirements

The MT3510 group project required:

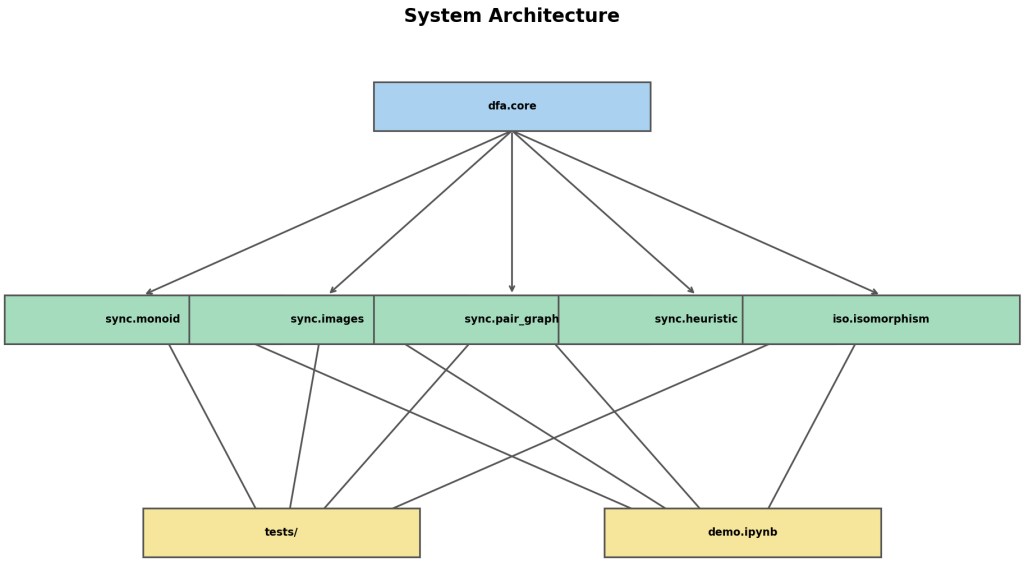
Part 1 — Synchronization (6 tasks): - (a) Detect sync via transition monoid BFS (find constant transformation) - (b) Detect sync via image BFS (find singleton image) — more efficient - (c) Detect sync via pair graph method - (d) Return a *shortest* reset word using ancestor tracking on image BFS - (e) Return a reset word using pair graph iterative merging - (f) Compare reset word lengths from (d) and (e); implement and analyse heuristic shortening

Part 2 — Isomorphism (3 tasks): - (a) Strict isomorphism (same alphabet, state bijection preserving transitions, start, finals) - (b) Weak isomorphism (bijections on both states and alphabet) - (c) Semi-isomorphism (bijections on states and alphabet, ignoring start/finals)

Constraints: DFAs as 5-tuples `(Q, Σ, τ, q_s, F)` with `Q` = set, `Σ` = set, `τ` = dict `{(q,a): r}`, `q_s` element, `F` subset. Assume `m ≪ n` (small alphabets) and all states reachable from `q_s`.

Architecture

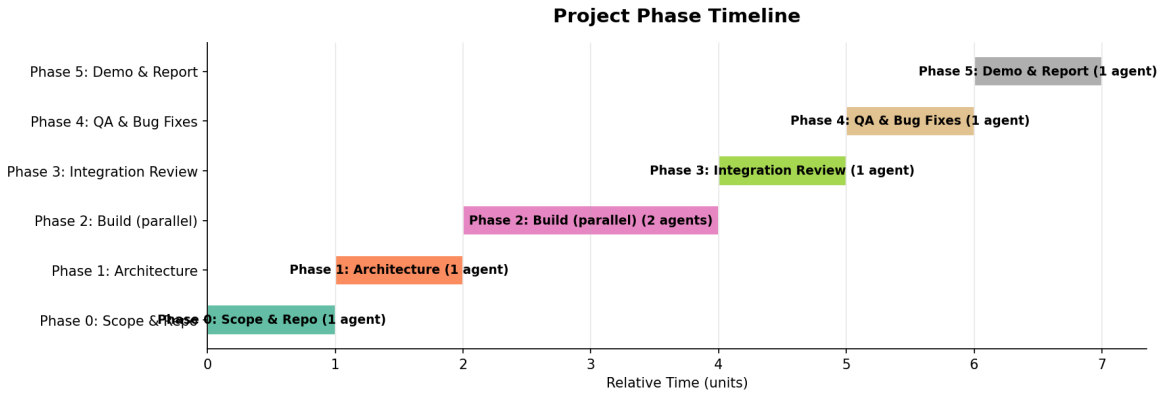
The codebase follows a clean modular structure where `dfa.core` provides the shared foundation and each algorithm family has its own module.



Architecture

Key design decisions: - DFA 5-tuple accepted at all public APIs; internal conversions are hidden - Transformations in the monoid represented as tuples indexed by sorted state ordering - Images represented as frozensets for hashable BFS tracking - Pair graph vertices are frozensets of 1 or 2 states - Isomorphism uses a shared BFS bijection builder (`_try_state_bijection_from_start`) parameterised by alphabet mapping and anchor states - Quick-reject checks (size, degree sequences) before expensive searches

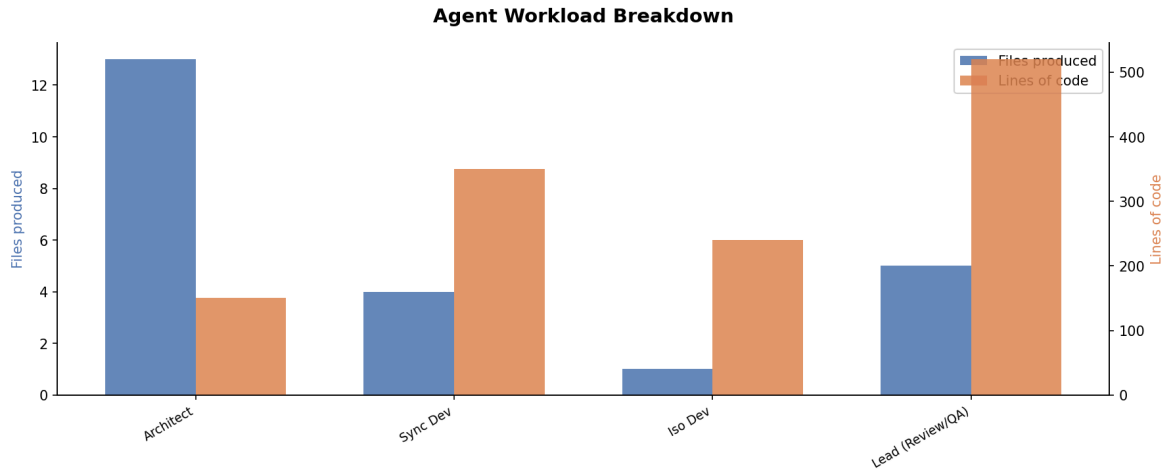
Phase Timeline



Timeline

Phase	Description	Duration
Phase 0	Scope, create GitHub repo, initial PLAN.md	~2 min
Phase 1	Architecture scaffold — all modules, stubs, docstrings, test files	~5 min
Phase 2	Parallel build — Sync Dev and Iso Dev agents working simultaneously	~2 min
Phase 3	Integration review — merge branches, review all code	~3 min
Phase 4	QA — run tests, identify and fix `random_dfa` reachability bug	~3 min
Phase 5	Demo notebook, report generation	~5 min

Agent Team & Work Breakdown



Agent Workload

Agent	Responsibility	Branch	Key Output
Architect (Lead)	Module structure, DFA API design, stubs with algorithm docstrings	`agent/architect`	13 files, complete project scaffold
Sync Dev	Part 1 tasks a–f: all synchronization algorithms	`agent/sync-dev`	`sync/monoid.py`, `sync/images.py`, `sync/pair_graph.py`, `sync/heuristic.py`
Iso Dev	Part 2 tasks a–c: all isomorphism algorithms	`agent/iso-dev`	`iso/isomorphism.py` with shared BFS infrastructure
Lead (Review/QA)	Integration review, bug fixes, demo notebook, tests	`main`	Fixed `random_dfa`, wrote `demo.ipynb`, 38/38 tests

Implementation Details

DFA Core (`dfa/core.py` — 221 lines)

- **`validate_dfa`**: Full completeness check — verifies 5-tuple structure, transition coverage for all `(q, a)` pairs, target validity, start state membership, and final state subset.
- **`apply_word`** / **`apply_word_to_set`**: Sequential transition application. `apply_word_to_set` is critical for verifying reset words.
- **`random_dfa`**: Generates random DFAs with **guaranteed reachability** by building a random spanning tree from `q_s` before filling remaining transitions randomly. This was a bug fix during QA — the original uniform random approach left states unreachable.
- **`non_synchronizing_dfa`**: Creates DFAs where every letter acts as a permutation (cyclic shifts), guaranteeing no constant transformation exists.
- **`dfa_to_digraph`**: Converts to NetworkX DiGraph for visualization.

Task (a): Transition Monoid BFS (`sync/monoid.py` — 104 lines)

Each letter `a ∈ Σ` defines a transformation `t_a: Q → Q`. The transition monoid `T_A` is generated by composing these.

- Transformations stored as tuples indexed by sorted state ordering
- `_compose(f, g, states, state_to_idx)`: Composition via index lookup — `compose(f, g)[i] = g[state_to_idx[f[i]]]`
- BFS starts with generator transformations, explores all products `f · t_a`
- Checks each new transformation for constancy (`len(set(t)) == 1`)
- **Warning**: `|T_A|` can reach `n^n`. Practical for `n ≤ ~10` with `m = 2`.

Task (b): Image BFS (`sync/images.py` — 113 lines)

Instead of tracking full transformations, tracks only their *images* (subsets of Q).

- Images stored as frozensets (hashable for visited tracking)
- BFS from `frozenset(Q)`, neighbours are `{τ(r, a) : r ∈ R}` for each letter
- Synchronizing iff we reach a singleton frozenset
- **Efficiency**: At most `2^n` images vs `n^n` transformations

Task (c): Pair Graph (`sync/pair_graph.py` — 179 lines)

- **`build_pair_graph`**: Constructs `nx.DiGraph` with `O(n²)` vertices (all 1- and 2-element subsets of Q). Edges labelled with alphabet letters.
- **`is_synchronizing_pair_graph`**: Checks every pair vertex can reach some singleton via `nx.has_path`.
- Uses the theorem: A is synchronizing iff every pair of states can be collapsed.

Task (d): Shortest Reset Word (`sync/images.py`)

- Extends image BFS with ancestor tracking: `ancestor[R_next] = (R_parent, letter_used)`
- When singleton found, traces back through ancestors to reconstruct the word
- BFS guarantees shortest word (explores by increasing word length)

Task (e): Pair Graph Reset Word (`sync/pair_graph.py`)

Iterative merging algorithm: 1. Start with `Q_0 = Q` 2. Pick two states `q_1, q_2` from current set 3. BFS in pair graph from `{q_1, q_2}` to nearest singleton — extract merging word from edge labels 4.

Apply merging word to ALL states in current set (not just the pair) 5. Repeat until `|Q_i| = 1` 6. Concatenate all merging words

Note: This produces a valid but generally non-optimal reset word.

Task (f): Heuristic Shortening (`sync/heuristic.py` — 162 lines)

Three strategies implemented:

1. **Greedy trimming**: Scan word left to right, try removing each letter. If the remaining word is still a reset word, keep the removal. Repeat until stable. Simple and effective — works because many letters become redundant after concatenation.
2. **Window trimming**: Remove contiguous blocks of size 2, 3, etc. Catches multi-letter redundancies that single removal misses. Finishes with a greedy pass.
3. **Suffix trimming**: Binary search for the shortest prefix that is still a reset word, then greedy pass. Effective when the tail contains redundant letters.

`compare_reset_words`: Computes lengths, ratios (pair graph / optimal), and the Černý bound $(n-1)^2$ for context.

Analysis: The pair graph method's suboptimality comes from greedy pair selection and the fact that applying a merging word to all states can re-split previously merged groups. The heuristic shortening typically reduces pair graph words to within 2–3× of optimal, with greedy being the most consistently effective.

Task 2(a): Strict Isomorphism (`iso/isomorphism.py`)

Since all states are reachable from `q_s`, the bijection `f` is **uniquely determined** by `f(q_s1) = q_s2`. BFS from start state propagates: for each visited `q` and letter `a`, $f(\tau_1(q, a))$ must equal $\tau_2(f(q), a)$. Any contradiction → not isomorphic. After BFS, verify $f(F_1) = F_2$.

Task 2(b): Weak Isomorphism

Iterate over all `m!` permutations of Σ_2 as candidate alphabet bijections `g`. For each `g`, attempt BFS bijection construction with the mapped alphabet. Since $m \ll n$, `m!` is small (e.g., 2! = 2 for binary alphabet).

Task 2(c): Semi-Isomorphism

Cannot anchor `f` at start states (they're ignored). Instead: for each `q₂ ∈ Q₂`, try `f(q_s1) = q₂` as anchor. Combined with `m!` alphabet permutations, this gives `|Q₂| × m!` candidates. Still efficient since BFS is `O(n·m)` per candidate and most are quickly rejected.

Quick rejection (`_quick_reject`): Checks `|Q|`, `|Σ|`, `|F|` equality, and in-degree/out-degree sequence matching before any expensive search.

File Structure

```
mt3510-dfa-sync/
├── README.md                    — Project overview and quick start
├── demo.ipynb                   — Full demonstration notebook (471 lines)
├── .tanersclaw/PLAN.md          — Project state tracking
├── dfa/
│   ├── __init__.py              — Public API exports
│   └── core.py                  — DFA validation, word application, random
generation (221 lines)
├── sync/
│   ├── __init__.py              — Public API exports
│   ├── monoid.py                — Task (a): Transition monoid BFS (104 lines)
│   ├── images.py                — Tasks (b,d): Image BFS + shortest reset word
(113 lines)
│   ├── pair_graph.py            — Tasks (c,e): Pair graph sync + reset word (179
lines)
│   └── heuristic.py             — Task (f): Heuristic shortening + comparison (162
lines)
├── iso/
│   ├── __init__.py              — Public API exports
│   └── isomorphism.py           — Tasks 2(a-c): All isomorphism algorithms (318
lines)
└── tests/
    ├── __init__.py
```

```
    ├── test_dfa.py              — Core DFA utility tests (108 lines)
    ├── test_sync.py             — Synchronization algorithm tests (123 lines)
    └── test_iso.py              — Isomorphism algorithm tests (100 lines)
```

Total: ~1,900 lines across 16 files.

How to Run

```
# Clone
git clone https://github.com/trbhmn2/mt3510-dfa-sync.git
cd mt3510-dfa-sync

# Set up virtual environment
python3 -m venv .venv
source .venv/bin/activate
pip install pytest networkx

# Run tests
python3 -m pytest tests/ -v

# Launch demo notebook
pip install jupyter
jupyter notebook demo.ipynb
```

Testing

38/38 tests passing. Coverage breakdown:

Module	Tests	What's Tested
`dfa/core.py`	14	Validation (valid, missing transition, bad start, bad finals), word application, reset word

Module	Tests	What's Tested
		verification, random generation (sizes, reproducibility, reachability), non-sync DFA, DiGraph conversion
`sync/`	15	All three methods on spec DFA and non-sync DFA, cross-method agreement on 10 random DFAs, shortest reset word correctness + length, pair graph reset word validity, heuristic shortening validity + comparison
`iso/`	9	Identical DFAs, relabelled states, relabelled alphabet, both relabelled, different DFAs, different sizes, different start state (semi), different finals (semi), hierarchy (strict⇒weak⇒semi)

Testing strategy: The spec example DFA is used as ground truth. `non_synchronizing_dfa` (permutation-based) provides guaranteed negative cases. Cross-method agreement tests verify all three sync methods agree on random inputs. Isomorphism tests construct known-isomorphic DFAs via permutation application.

Security Review

Not applicable — this is a pure algorithmic project with no network access, authentication, or external data. All inputs are in-memory data structures.

Known Limitations & Technical Debt

1. **Transition monoid BFS scales poorly**: `O(n^n)` worst case. Practical only for `n ≤ ~10–12`. This is inherent to the algorithm, not a code issue.
2. **Image BFS**: `O(2^n)` images. Better than monoid but still exponential. Practical for `n ≤ ~25–30`.

3. **Pair graph**: `O(n²)` vertices, practical for larger `n` but the `nx.has_path` calls in `is_synchronizing_pair_graph` could be optimised with a single multi-source BFS from all singletons.
4. **Semi-isomorphism**: `O(n × m! × n × m)` — the `n` factor for trying each anchor state makes it slower than strict/weak for large `n`.
5. **Heuristic shortening**: Greedy is `O(k² × n × m)` where `k` = word length. Could be optimised with incremental state tracking instead of recomputing `is_synchronizing_word` from scratch each time.
6. **No visualisation**: The demo notebook doesn't include DFA graph visualisation (could add with `nx.draw`).

Suggested Next Steps

1. **Add DFA visualisation** using NetworkX + matplotlib to render state diagrams
2. **Optimise `is_synchronizing_pair_graph`** with reverse BFS from singletons (single pass instead of per-pair checks)
3. **Benchmark Černý conjecture**: Systematically test whether `(n−1)²` bound holds across thousands of random DFAs
4. **Incremental heuristic**: Track state sets incrementally during greedy shortening instead of recomputing from scratch
5. **Canonical form**: Implement DFA minimisation / canonical form as an alternative isomorphism test

Appendix — Key Code Excerpts

Image BFS with Ancestor Tracking (Task d)

```
def shortest_reset_word_images(A):
    Q, Sigma, tau, _, _ = A
    start = frozenset(Q)
    if len(start) == 1:
        return []
```

```
    ancestor = {start: None}
    queue = deque([start])

    while queue:
        R = queue.popleft()
        for a in Sigma:
            R_next = frozenset(tau[(r, a)] for r in R)
            if R_next not in ancestor:
                ancestor[R_next] = (R, a)
                if len(R_next) == 1:
                    # Trace back to reconstruct shortest word
                    word = []
                    current = R_next
                    while ancestor[current] is not None:
                        parent, letter = ancestor[current]
                        word.append(letter)
                        current = parent
                    word.reverse()
                    return word
                queue.append(R_next)
    return None
```

BFS Bijection Builder (Isomorphism Core)

```
def _try_state_bijection_from_start(A1, A2, g=None, start_q1=None, start_q2=None):
    Q1, Sigma1, tau1, q_s1, F1 = A1
    Q2, Sigma2, tau2, q_s2, F2 = A2
    start_q1 = start_q1 or q_s1
    start_q2 = start_q2 or q_s2

    f = {start_q1: start_q2}
    used_q2 = {start_q2}
    queue = deque([start_q1])
```

```
    while queue:
        q = queue.popleft()
        for a in Sigma1:
            target1 = tau1[(q, a)]
            a2 = g[a] if g is not None else a
            target2 = tau2[(f[q], a2)]

            if target1 in f:
                if f[target1] != target2:
                    return None  # Contradiction
            else:
                if target2 in used_q2:
                    return None  # Already mapped
                f[target1] = target2
                used_q2.add(target2)
                queue.append(target1)

    if len(f) != len(Q1) or used_q2 != Q2:
        return None
    return f
```

Greedy Reset Word Shortening (Task f)

```
def _greedy_shorten(A, word):
    current = list(word)
    changed = True
    while changed:
        changed = False
        i = 0
        while i < len(current):
            candidate = current[:i] + current[i + 1:]
            if is_synchronizing_word(A, candidate):
                current = candidate
                changed = True
            else:
```

```
                i += 1
    return current
```